COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	8/28/26	**DATE SUBMITTED:**	8/28/26
NAME:	Gabriel Dwyane D. Aridon	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO

1. PROCEDURE OUTPUT

Linear Search (Array)

Header

```
#include "node.h"


#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H


template<typename T>
void linearSearchAR(int size, T dataFind, T array[]) {

    for (int j = 0; j < size; j++) {
        if(dataFind == array[j]) {
            std::cout << "Searching is successful\n";
            return;
        }

    }
    std::cout << "Searching is unsuccessful\n";
}
```

Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    int dataset[max_size];

    srand(time(0));

    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand();
        std::cout << dataset[i] << " ";
    }

    std::cout << std::endl;

    std::cout << "What number to find: ";
    int choice;

    std::cin >> choice;

    linearSearchAR(max_size, choice, dataset);

    return 0;
}
```

Output

```
14237 20247 17333 20453 23468 7174 4569 9708 31354 13869 21493 20778 14926 28484 1839 17209 19241 26847 29216 26209 29699 31902 15414 26472 1368 29860 5206 20906 21731 10315 10950 12652 9898 20758 587 12397
 910 3036 23041 7195
What number to find: 1
Searching is unsuccessful
```

```
14015 10252 14986 22547 10205 2300 22299 12155 22678 5953 31550 17783 16811 17446 100 17867 30625 19116 3031 32163 7109 27774 10213 8231 24174 31527 30785 14671 30338 38
7 3494 17145 12859 8563 7357 4153 1323 31189 7233 752 14206 10420 3681 16288 6784 15803 31078 21374 26207 12528
What number to find: 12528
Searching is successful
```

Observation

When we typed in 1, the program printed “Searching is unsuccessful” because none of the randomly generated numbers matched that input. But when you typed in 12528, the program printed “Searching is successful” since that exact number happened to be on that random array.

Linear Search (Linked List)

Header

```
template<typename T>
void linearSearchLL(Node<T> *nodes, T dataFind) {

while(nodes != nullptr){

    if(nodes->data == dataFind){
        std::cout << "The data: " << nodes->data << " was found successfully.\n";
        return;
        }
    nodes = nodes->next;

    }
    std::cout << "Searching is unsuccessful.\n";
}
```

Main

Linear Search LL > linearLL.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "searchingAlgo.h"

int main() {
    Node<char> *char1 = newNode('G');
    Node<char> *char2 = newNode('A');
    Node<char> *char3 = newNode('B');
    Node<char> *char4 = newNode('R');
    Node<char> *char5 = newNode('I');
    Node<char> *char6 = newNode('E');
    Node<char> *char7 = newNode('L');


    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = char5;
    char5->next = char6;
    char6->next = char7;

    linearSearchLL(char1, 'L');
    linearSearchLL(char1, 'S');

}
```

Output

```
The data: L was found successfully.
Searching is unsuccessful.
```

Observation

When we call the linearSearchLL(char1, 'L') function, the program walked through the list node by node until it hit the last one, which held 'L'. That's why it printed "The data: L was found successfully." Since 'S' doesn't exist in any of the nodes, the search reached the end of the list without a match, so it printed "Searching is unsuccessful."

Binary Search (Array)

Header

```
template<typename T>
void binarysearchArr(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize - 1, middle;
    while(low <= high){
        middle = (int)((low + high / 2));
        if(dataFind == data[middle]){
            std::cout << "Data was found.\n";
            return;
        }
        else if(dataFind < data[middle]){
            high = middle - 1;

        }
        else{
            low = middle + 1;
        }
    }
    std::cout << "Data was not found\n";


}
```

Main

```
    int newArr[10] = {1,2,3,4,5,6,7,8,9,10};

    for(int i = 0; i < max_size; i++) {
        newArr[i] = i;
    }

    binarysearchArr(newArr, 10, 11);

    return 0;
}
```

```
int newArr[10] = {1,2,3,4,5,6,7,8,9,10};

for(int i = 0; i < max_size; i++) {
    newArr[i] = i;
}

binarysearchArr(newArr, 10, 1);

return 0;
}
```

Output

```
Data was not found
```

```
Data was found.
```

Observation

When searched for 11, the program printed “Data was not found” because the array only held values from 0–9 after the loop, so 11 simply wasn’t there. But when searched for 1, the program printed “Data was found” since 1 exists in that same range.

Binary Search (Linked List)

Header

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    //set start to the head of the list and set last to null
    Node<T>* last = nullptr;

    while(start != last){

        Node<T> *middle = getMiddle(start, last);

        // If the middle element is same as the key to be searched, we return it.
        if(middle->data == dataFind){
            std::cout << "Searching is successful\n";
            return;
        }
        // Else if middle element is greater than the key to be searched,
        // we have to search in the left side of the singly linked list.
        else if(middle->data > dataFind){
            last = middle;
        }
        // Else if middle element is less than the key to be searched,
        // we have to search in the right side of the singly linked list.
        else{
            start = middle->next;
        }
    }

    std::cout << "Searching is unsuccessful\n";
}
```

```cpp
template <typename T>
Node<T>* getMiddle(Node<T> *start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    //continue until fast reaches the end or the last node of the current search range
    while(fast != last && fast->next != last) {

        //move slow one step
        slow = slow->next;
        //move fast two steps
        fast = fast->next->next;
    }

    return slow;
}
```

Main

```
// ===== Binary Search Linked List =====

    Node<char> *char1 = newNode('G');
    Node<char> *char2 = newNode('A');
    Node<char> *char3 = newNode('B');

    char1->next = char2;
    char2->next = char3;
    char3->next = nullptr;

    binarySearchLL(char1, 'B');
    binarySearchLL(char1, 'G');

    return 0;
}
```

Output

```
Searching is successful
Searching is unsuccessful
```

Observation

When searched for 'B', the program printed “Searching is successful” because the algorithm’s path happened to land right on it. But when you searched for 'G', even though it’s sitting at the head of the list, the binary search logic skipped over it since the list wasn’t sorted. That’s why the output shows “Searching is unsuccessful”.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

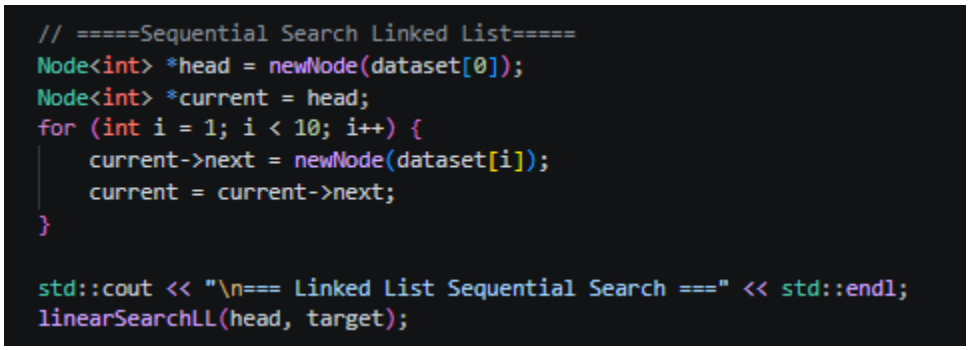
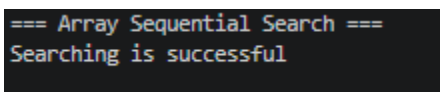
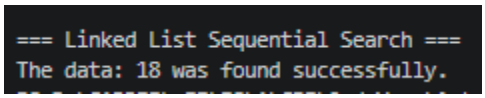
ILO B: Solve different problems utilizing appropriate searching techniques in C++

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key ‘18’?

Code

```
    int dataset[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int target = 18;

    // ====Sequential Search Array=====
    std::cout << "=== Array Sequential Search ===" << std::endl;
    linearSearchAR(10, target, dataset);
```

	
Output	 
Observation	Both the array and linked list need exactly two comparisons. This is because sequential search checks each element one by one, starting from the beginning. It finds the first 18 at the second position, so it stops right away and does not check the other 18 later in the list.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

Code

```
template<typename T>
int countSequentialAR(int size, T dataFind, T array[]) {
    int count = 0;
    for (int j = 0; j < size; j++) {
        if (dataFind == array[j]) {
            count++;
        }
    }
    return count;
}

template<typename T>
int countSequentialLL(Node<T> *nodes, T dataFind) {
    int count = 0;
    Node<T> *current = nodes;
    while (current != nullptr) {
        if (current->data == dataFind) {
            count++;
        }
        current = current->next;
    }
    return count;
}
```

main

```
int dataset[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
int target = 18;
```

```
// =====Sequential Search Linked List=====
Node<int> *head = newNode(dataset[0]);
Node<int> *current = head;
for (int i = 1; i < 10; i++) {
    current->next = newNode(dataset[i]);
    current = current->next;
}
```

```
int arcount = countSequentialAR(10, target, dataset);
int llcount = countSequentialLL(head, target);
std::cout << "Array Count for key '" << target << "': " << arcount << std::endl;
std::cout << "Linked List Count for key '" << target << "': " << llcount  << std::endl;
```

Output

```
Array Count for key '18': 2
Linked List Count for key '18': 2
```

Observation

Both array and linked list algorithms accurately detect 2 occurrences of the target key ‘18’ the one at index 1 and index 4

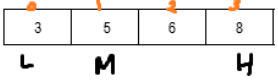
Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Iteration: 1



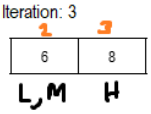
Target 8 is less than mid value (11) -> so target should be in left half
Set high = mid - 1 (so index 4 - 1 = 3)

Iteration: 2



Target 8 is greater than mid value (5) -> target should be in right half
Set low = mid + 1 (2)

Iteration: 3



Target 8 is greater than mid value (6) -> target should be in right half
Set low = mid + 1 (3)

Iteration: 4



Target 8 == Mid Value (8) -> Match is found at index 3

Code

```
template<typename T>
void binSearchArr(T data[], int arrSize, T dataFind) {
    int low = 0, high = arrSize - 1, middle;
    int i= 1;
    while (low <= high) {
        middle = (int)((low + high / 2));
        std::cout << "Iteration " << i << ": low=" << low << " [" << data[low]
                  << "], high=" << high << " [" << data[high]
                  << "], middle=" << middle << " [" << data[middle] << "]\n";

        if (dataFind == data[middle]) {
            std::cout << "Key " << dataFind << " successfully found at index " << middle << ".\n";
            return;
        } else if (dataFind < data[middle]) {
            high = middle - 1;
        } else {
            low = middle + 1;
        }
        i++;
    }
    std::cout << "Key " << dataFind << " was not found.\n";
}
```

```
//============Problem 3============
int sortedData[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int target1 = 8;
binSearchArr(sortedData, 10, target1);
```

Output

```
Iteration 1: low=0 [3], high=9 [18], middle=4 [11]
Iteration 2: low=0 [3], high=3 [8], middle=1 [5]
Iteration 3: low=2 [6], high=3 [8], middle=3 [8]
Key 8 successfully found at index 3.
```

Observation	The binary search found the target 8 in only 4 steps from 10 elements. It works by checking the middle and narrowing the search to either the left or right side until the target is found.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code

```
template<typename T>
int binarySearchRecursiveAR(T array[], int low, int high, T dataFind, int depth = 1) {
    // Base Case 1: Target not in list
    if (low > high) {
        std::cout << "Key " << dataFind << " was not found.\n";
        return -1;
    }

    int middle = low + (high - low) / 2;
    std::cout << "Recursion Call " << depth << ": low=" << low
              << " [" << array[low] << "], high=" << high
              << " [" << array[high] << "], middle=" << middle
              << " [" << array[middle] << "]\n";

    // Base Case 2: Target found
    if (dataFind == array[middle]) {
        std::cout << "Key " << dataFind << " successfully found at index " << middle << ".\n";
        return middle;
    }
    // Recursive Step: Search left half
    else if (dataFind < array[middle]) {
        return binarySearchRecursiveAR(array, low, middle - 1, dataFind, depth + 1);
    }
    // Recursive Step: Search right half
    else {
        return binarySearchRecursiveAR(array, middle + 1, high, dataFind, depth + 1);
    }
}
```

```
int sortedData[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int target1 = 8;
```

```
binarySearchRecursiveAR(sortedData, 0, 9, target1);
```

Output

```
Recursion Call 1: low=0 [3], high=9 [18], middle=4 [11]
Recursion Call 2: low=0 [3], high=3 [8], middle=1 [5]
Recursion Call 3: low=2 [6], high=3 [8], middle=2 [6]
Recursion Call 4: low=3 [8], high=3 [8], middle=3 [8]
Key 8 successfully found at index 3.
```

Observation

The recursive binary search also found 8 at index 3 in 4 steps, giving the same result as Problem 3. Instead of using a loop, it calls itself with a smaller part of the list until the target is found.

Conclusion: Linear search works well with both arrays and linked lists because it simply checks each element one at a time from the beginning. Binary search works better with arrays because you can quickly jump to the middle, while linked lists require you to go through each node to reach it. Through these exercises, I learned how sequential and binary searching algorithms operate using arrays and linked lists. Sequential search is straightforward and works on unsorted data, but finding all repeating instances requires inspecting every single item in the list. Binary search is significantly faster because it eliminates half of the dataset at each step, though it strictly requires the list to be sorted first.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used assistance of AI specifically for problem 4 since at first I don't understand how recursive works and how to apply it in binary search.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.